CP2406

Assessment Task 3, Option 3 Hint Guide

Barnes-Hut Galaxy Simulation | Balanced Scaffold Edition

Purpose This guide gives staged hints for the supplied Galaxy project without providing a complete solution students can submit unchanged. Use it after reading the task sheet and attempting each task.

Before You Start

- Build and run the supplied project before changing it. Confirm that frames, CSV output and the optional video pathway work in the development environment.
- Keep the original Barnes-Hut force calculation working. The assignment focuses on configuration, collision handling, reporting, testing and software organisation.
- Use version control or regular backups. Change one responsibility at a time and test after each change.
- Sketch the flow: command-line arguments -> validated configuration -> body initialisation -> simulation steps -> collision pass -> metrics -> CSV, frames and video.

Recommended order Validate configuration first. Then unit-test a two-body merge, test the bounded sampling pass on a tiny vector, connect it to the simulation, and only then run full experiments.

Where to Work in the Supplied Project

File or component	Main responsibility	Student focus
SimulationConfig.h/.cpp	Configuration defaults, command-line parsing and validation	Check every required parameter and help behaviour.
Collisions.h/.cpp	Merge operation, bounded random sampling and statistics	Test self-pairs, retired bodies, distance threshold, conservation and termination.
Simulation.h/.cpp	Initialisation, simulation steps and summary metrics	Keep the force pathway working and coordinate

		collision results.
Renderer.h/.cpp	PPM snapshot output	Keep rendering separate from simulation logic.
CsvReporter.h/.cpp	Experiment-row output	Ensure required fields, failure handling and deterministic experiment ordering.
main.cpp	Top-level coordination	Keep thin: parse, run, report, render/video and handle errors.
Bhtree / Octant / Types	Existing tree, spatial and data structures	Review pointer and boundary behaviour; avoid unnecessary physics redesign.

Naming check The supplied header is named simulation.h while source files include Simulation.h on a case-sensitive system. Use one consistent capitalisation throughout the project.

Task 1: Configurable Simulation and Investigation

Task 1 function map: SimulationConfig.h -> SimulationConfig data members and defaults; SimulationConfig.cpp -> SimulationConfig::validate(), parseArguments(...), and helpText(). The controlled experiment runs are coordinated from main() or a separate experiment helper.

Hint 1: Keep settings together

Where this belongs: SimulationConfig.h -> SimulationConfig fields and defaults; SimulationConfig.cpp -> parseArguments(int argc, char** argv). Parse options here, assign values to the configuration object, and call config.validate() after parsing.

- Place user-adjustable values in SimulationConfig rather than adding macros to Constants.h.
- Give each setting a sensible default, then let command-line options override selected values.
- After parsing, call one validation function so defaults and command-line values follow the same rules.

```
for each command-line argument:
    recognise the option
    confirm that a value follows when one is required
    convert the value safely
    reject unknown options
after parsing:
    validate the complete configuration
```

Hint 2: Validate meaning, not only conversion

Where this belongs: SimulationConfig.cpp -> SimulationConfig::validate() const for complete parameter checks; parseArguments(...) for missing values, conversion failures, unknown options and --help handling; helpText() for the displayed option list.

- Body count must support the operations the program performs.
- System size and time step must be positive; step count must permit at least one simulation step.
- Collision threshold cannot be negative. Decide and document whether zero disables merging or permits only zero-distance merges.
- Validate collision sample count explicitly, including the intended meaning of zero attempts.
- Conversion functions can throw. Present a clear message rather than continuing with a partially changed configuration.

Help behaviour --help should show the supported options without starting a simulation. Keep help handling distinct from an invalid failure where practical.

Hint 3: Conduct a controlled investigation

Where this belongs:main.cpp -> main() or a new helper such as runExperimentMatrix(...). Create each SimulationConfig, initialise a fresh body vector, call runExperiment(...), and pass each ExperimentResult to the reporting pathway.

- Change at least three parameters while holding the remaining baseline settings fixed.
- Use a stable experiment name and explicit seed so rows can be compared and sorted predictably.
- Record the configuration before discussing visible or measured effects. Include one invalid case and its message.

Experiment	Changed parameter	Fixed settings	Observed evidence
Invalid case			

Task 2: Collision Detection and Safe Merging

Task 2 function map: Collisions.cpp -> checkAndMergeCollision(...) for one pair; Collisions.cpp -> sampleAndMergeCollisions(...) for bounded random sampling and CollisionStats.

Hint 1: Test one merge independently

Where this belongs: Collisions.cpp -> checkAndMergeCollision(body& a, body& b, double thresholdAU). Put threshold, self-pair and retired-body checks first, then distance, conserved merge calculations and retirement of the second body.

- Reject a negative threshold before calculating distance.
- Return false for the same object and for a body already retired according to the project convention.
- Use the Euclidean distance between positions measured in the same units as thresholdAU.
- If the bodies are within the threshold, compute total mass before changing either body.
- Calculate merged position and velocity from the original masses and values, then update the survivor and retire the second body.

```
totalMass = massA + massB
mergedPosition = (positionA * massA + positionB * massB) / totalMass
mergedVelocity = (velocityA * massA + velocityB * massB) / totalMass
```

Conservation check Before and after a merge, compare total mass and total linear momentum. Also calculate the expected centre-of-mass position manually for a simple two-body case.

Hint 2: Keep the sampling loop bounded

Where this belongs: Collisions.cpp -> sampleAndMergeCollisions(std::vector<body>& bodies, double thresholdAU, std::size_t attempts, std::uint32_t seed). The attempt-limited loop, random index selection, valid-pair checks, merge count and final active-body count all belong here.

- The requested sample count is the maximum number of attempts, not the number of successful merges.
- Create std::mt19937 from the supplied seed and draw indices from the valid vector index range.
- If both indices are equal, count the attempt but do not attempt a self-merge.
- Only count a valid pair after the indices are distinct and both selected bodies are active.
- After all attempts, count remaining active bodies once and store the result in CollisionStats.

```
repeat exactly attempts times:
    attemptedPairs += 1
    select indexA and indexB
    if same index: continue
    if either body is retired: continue
    validPairs += 1
    if merge succeeds: mergedPairs += 1
```

Review the supplied code carefully Confirm that validPairs really excludes retired-body pairs. Calling the merge function and receiving false is not necessarily the same as selecting a valid pair.

Task 3: Reporting, Refactoring and Reproducibility

Task 3 function map: Simulation.cpp -> initialiseBodies(...), simulation-step helper and runExperiment(...); Renderer.cpp -> renderSnapshot(...); CsvReporter.cpp -> writeExperimentCsv(...); main.cpp -> top-level coordination and experiment ordering.

Hint 1: Separate responsibilities

Where this belongs: Simulation.cpp -> initialiseBodies(...), internal simulateStep(...), summary helper and runExperiment(...); Renderer.cpp -> renderSnapshot(...); CsvReporter.cpp -> writeExperimentCsv(...); main.cpp -> parse, coordinate, report progress, create video and handle top-level errors.

- Simulation should own initialisation, stepping and summary calculations.
- Renderer should turn active body positions into image output.
- CsvReporter should serialise experiment data, not run the experiment.
- main.cpp should parse, coordinate, report progress and handle top-level errors.

Hint 2: Verify every CSV field

Where this belongs: CsvReporter.cpp -> writeExperimentCsv(...). File-open checking, header creation, column order and row writing belong here. If deterministic ordering across several experiments is required, collect and sort rows in main() or a separate reporting helper before writing.

- Include configuration values, seed, elapsed time, attempted/valid/merged counts, active bodies, total mass, barycentre coordinates and status.
- Check that the output file opened successfully before writing.
- Write the header once and keep the row-column order identical to the header.
- If failed experiments must be reported, do not hard-code every status as ok.
- Deterministic ordering may require collecting rows and sorting them before writing, rather than relying only on append order.

Hint 3: Plan the required experiment matrix

Where this belongs:main.cpp -> main() or a new runExperimentMatrix(...) helper. Create fresh bodies for every configuration, keep baseline values controlled, repeat at least one identical seeded configuration, then sort and report the resulting experiment rows deterministically.

Body count	Threshold	Seed	Repeat?	Evidence to

				compare
Setting A	Disabled or zero			
Setting A	Rare			
Setting A	Higher			
Setting B	Disabled or zero			
Setting B	Rare			
Setting B	Higher			
Repeat one row	Same as original	Same seed	Yes	Compare deterministic results

- Compare runtime, active-body count, total mass and barycentre rather than relying only on the video.
- Repeating a seeded configuration should use the same starting configuration and seed. Treat elapsed wall-clock time separately because timing can vary between runs.
- Explain visible output cautiously. A visual difference is evidence to discuss, not proof of a physical conclusion.

Task 4: Code Review and Testing

Task 4 function map:A separate test source file is recommended, for example GalaxyTests.cpp, with focused helpers for configuration validation, checkAndMergeCollision(...) and sampleAndMergeCollisions(...). CODE_REVIEW.md remains a separate written deliverable.

Guiding prompts for CODE_REVIEW.md

- Explain pointer member access by rewriting a->position.x using dereference syntax.
- Trace insertIntoChild in its actual else-if order and compare that with Octant::contains at shared boundaries.
- Explain why selecting the same vector element twice could corrupt a merge.
- Explain why waiting for a fixed number of successful collisions can fail to terminate.
- Use the centre-of-mass and momentum equations to explain what a correct merge preserves.
- Explain how an explicit mt19937 seed supports repeatable random selections.
- Name the responsibilities moved from main.cpp and link each move to a testability benefit.
- For every test, state input, expected result, actual result and conclusion.

Write in your own words Refer to specific functions and lines in the submitted project. Do not copy the task pseudocode or this hint guide as the review answer.

Focused Test Checklist

Where these tests belong: Prefer a separate test file and test executable. Suggested test functions include testConfigurationValidation(), testTwoBodyMerge(), testCollisionBoundaries(), testRetiredAndSelfPairs(), testBoundedSampling(), and testSeededReproducibility(). If no test framework is used, a clearly separated manual test driver is acceptable evidence.

- ☐ Negative collision threshold is rejected.
- ☐ Same-body reference returns false and does not change mass, position or velocity.
- ☐ Retired body is ignored.
- ☐ Distance greater than threshold does not merge.
- ☐ Distance exactly equal to threshold follows the documented boundary rule.
- ☐ Two simple bodies merge with expected mass, centre of mass and velocity.
- ☐ Zero attempts terminates with zero attempted pairs and a correct active count.
- ☐ One-body or empty collection terminates safely.
- ☐ Same seed and starting data produce the same sampled outcomes.
- ☐ Zero/negative configuration values and missing command-line values are handled clearly.

Common Problems to Diagnose

Files and functions to inspect: Configuration problems: SimulationConfig.cpp. Merge or statistics problems: Collisions.cpp. Mass, barycentre or step problems: Simulation.cpp. CSV problems: CsvReporter.cpp. Frame problems: Renderer.cpp. FFmpeg and top-level output problems: main.cpp. Case-sensitive include problems: Simulation.h/simulation.h and matching #include statements.

Symptom	What to inspect
Zero merges in every run	Threshold scale, body distances, retired-body convention, sample count and whether collisions are disabled.
Program appears to hang	Check for loops bounded by successful merges rather than attempts, or excessive rendering/step settings.
Mass changes after merging	Ensure original masses are saved before updating the survivor and retired bodies are excluded only after their mass is transferred.
Barycentre changes unexpectedly at merge	Check centre-of-mass position and that both original masses and positions are used.
validPairs is unexpectedly high	Ensure retired selections are excluded before incrementing validPairs.
Seeded repeat differs	Confirm identical starting configuration, seed, experiment order and no shared mutated body

	vector.
CSV columns shift	Compare each emitted value with the header order and check output opening.
Build works on one OS only	Check Simulation.h/simulation.h capitalisation and generated-file paths.
Video is not produced	Frames may still be valid; verify FFmpeg command result and input naming separately.

Final Submission Check

- ☐ Clean CLion build succeeds and the original Barnes-Hut simulation still runs.
- ☐ --help, required command-line options and invalid input handling are demonstrated.
- ☐ Collision sampling is bounded and statistics use the documented definitions.
- ☐ CSV contains all required fields and the required experiment matrix.
- ☐ At least four meaningful tests include edge and reproducibility cases.
- ☐ CODE_REVIEW.md answers all eight questions and includes a verified AI-use statement.
- ☐ ZIP excludes build folders, IDE caches, generated frame sequences, temporary files and unnecessary videos.
- ☐ Video shows a valid run, invalid configuration, collision statistics, CSV evidence, refactoring and one reproducibility test.

Last advice Debug the collision algorithm with manually created bodies before running the full galaxy. Then use a tiny fixed-seed sampling case. This isolates logic errors from Barnes-Hut tree behaviour, rendering and long experiment runs.

Guided Design Challenges and Selected Coding Scaffolds

Balanced approach: partial code is retained for configuration validation, command-line traversal, statistics initialisation, CSV output and manual tests. Core collision merging, bounded sampling, deterministic experiment ordering and responsibility separation are explanation-led design challenges.

Scaffold 1: Validate the Complete Configuration

Where this belongs: SimulationConfig.cpp -> SimulationConfig::validate() const

```
if (bodyCount < ___________) {
    throw std::invalid_argument("Body count is too small.");
}
if (systemSizeAU ___________ 0.0 || timeStep ___________ 0.0) {
    throw std::invalid_argument(
        "System size and time step must be positive.");
}
```

```
if (stepCount < __________) {
    throw std::invalid_argument("At least one step is required.");
}
if (collisionThresholdAU __________ 0.0) {
    throw std::invalid_argument(
        "Collision threshold cannot be negative.");
}
```

Decide and document the minimum body count and the meaning of a zero collision threshold. Do not add checks that contradict the task sheet.

Scaffold 2: Traverse Command-Line Arguments Safely

Where this belongs: SimulationConfig.cpp -> parseArguments(int argc, char** argv)

```
for (int index = 1; index < __________; ++index) {
    const std::string option = __________;
    if (option == "--help") {
        __________;
    }
    if (option == "--bodies") {
        if (index + 1 >= argc) {
            throw std::invalid_argument("Missing value for --bodies.");
        }
        config.bodyCount = convertBodyCount(__________);
        ++index;
    }
    // Handle the remaining required options and unknown options.
}
config.__________();
```

The scaffold shows traversal only. Students must design conversion-error handling, all required options, help behaviour and unknown-option reporting.

Design Challenge 3: Implement a Safe Two-Body Merge

Where this belongs: Collisions.cpp -> checkAndMergeCollision(body& a, body& b, double thresholdAU)

Implement this function without supplied code.

The design must:
- reject a negative threshold;
- return false for a self-pair or either retired body;
- compare distance using consistent units;
- preserve both original masses, positions and velocities until all merged values are calculated;
- reject or safely handle a non-positive combined mass;
- update the survivor using centre-of-mass position and momentum-preserving velocity;
- retire the second body using the project convention; and
- return true only when a merge occurs.

Before coding, calculate a simple unequal-mass example by hand. State the expected total mass, position, velocity and retired-body state.

Design Challenge 4: Build a Bounded Collision Pass

Where this belongs: Collisions.cpp -> sampleAndMergeCollisions(...)

Design the attempt-limited loop yourself.

Your implementation must distinguish attempted pairs, valid pairs and merged pairs. A valid pair has distinct indices and two active bodies. The loop must execute no more than the requested number of attempts, even when no collision can occur.

Decide and explain:
• behaviour for an empty or one-body vector;
• the valid index range used by uniform_int_distribution;
• when each CollisionStats counter is incremented;
• why a failed merge can still follow a valid pair selection;
• how the explicit seed controls repeatability; and
• how active bodies are counted after sampling.

Trace three attempts manually: a self-pair, a pair containing a retired body, and a distinct active pair outside the threshold.

Scaffold 5: Initialise and Finalise Collision Statistics

Where this belongs: Collisions.cpp -> sampleAndMergeCollisions(...)

```
CollisionStats stats{};
if (bodies._____________()) {
    return stats;
}
// Run the bounded sampling logic designed in Challenge 4.
stats.activeBodies = static_cast<std::size_t>(
    std::count_if(
        bodies.begin(), bodies.end(),
        [](const body& value) {
            return _____________;
        }));
return _____________;
```

Use the same retired-body convention throughout merging, sampling, rendering and summary calculations.

Design Challenge 6: Separate Simulation Responsibilities

Where this belongs: Simulation.cpp, Renderer.cpp, CsvReporter.cpp and main.cpp

Create a responsibility plan before moving code.

For each existing block in main.cpp, classify it as one of:
• configuration/parsing;
• body initialisation;
• one simulation step;
• collision processing;
• metric calculation;
• rendering;
• CSV serialisation; or
• top-level coordination/error handling.

Then decide the destination function and its inputs/outputs. Explain how the move enables a focused test. main.cpp should coordinate the workflow rather than own physics, rendering or serialisation

details.

Design Challenge 7: Deterministic Experiment Matrix and Reproducibility

Where this belongs: main.cpp or a student-created runExperimentMatrix(...) helper

Design the experiment collection and ordering without supplied code.

Your solution must create fresh starting bodies for each configuration, store the explicit seed, run the required threshold/body-count combinations, repeat one identical seeded configuration, and sort result rows deterministically before reporting.

Document:
• the baseline settings held fixed;
• the comparison keys used to order rows;
• which outputs should repeat exactly for an identical seed and starting state;
• why elapsed wall-clock time may differ; and
• how failed experiments are represented without losing the configuration that caused the failure.

Scaffold 8: Write One Experiment CSV Row

Where this belongs: CsvReporter.cpp -> writeExperimentCsv(...)

```
for (const ExperimentResult& result : results) {
    output << result._____________ << ','
           << result.config._____________ << ','
           << result._____________ << ','
           << result.elapsedSeconds << ','
           << result.collisionStats._____________ << ','
           << result.collisionStats._____________ << ','
           << result.collisionStats._____________ << ','
           << result._____________ << ','
           << result.totalMass << ','
           << result.barycentre.x << ','
           << result.barycentre.y << ','
           << result.barycentre.z << ','
           << result._____________ << '\n';
}
```

Match every expression to the CSV header. The exact field names depend on the student's structures, so the row must not be copied without checking the project design.

Scaffold 9: Small Manual Merge Test

Where this belongs: GalaxyTests.cpp or a clearly separated temporary test helper

```
body first = makeTestBody(
    /* mass */ 2.0,
    /* position */ {0.0, 0.0, 0.0},
    /* velocity */ {1.0, 0.0, 0.0});
body second = makeTestBody(
    /* mass */ _____________,
    /* position */ {_____________, 0.0, 0.0},
    /* velocity */ {_____________, 0.0, 0.0});

const bool merged = checkAndMergeCollision(
    first, second, _____________);
```

```
// Calculate before running:
// expected merged mass = ___________
// expected x position  = ___________
// expected x velocity  = ___________
// expected retired state of second = ___________
```

Create separate tests for a self-pair, retired body, distance above threshold, exact-threshold distance and non-positive total mass.

Design and Implementation Checklist

- ☐ Configuration defaults and command-line overrides use the same validation rules.
- ☐ Help exits without starting the simulation.
- ☐ Merge calculations use original values and preserve mass and linear momentum.
- ☐ Sampling is bounded by attempts and cannot wait for successful collisions.
- ☐ Same-index and retired-body selections are not counted as valid pairs.
- ☐ Identical seeds and starting data reproduce sampled outcomes.
- ☐ Experiment rows are ordered deterministically and failed runs retain useful status.
- ☐ CSV values follow the header order exactly.
- ☐ main.cpp coordinates rather than containing simulation, rendering and reporting logic.
- ☐ Every scaffold completion and design decision can be explained and modified.

> ***Academic integrity: the retained scaffolds support syntax-heavy tasks but omit central algorithmic decisions. The design challenges require students to develop, test and explain their own implementation.***

CP2406 Assessment Task 3 | 2026 | Option 3 Galaxy Balanced Scaffold Hint Guide